



Complexity Theory Recap

22 April 2025

Prof. Dr. Sebastian Wild

Outline

2 Complexity Theory Recap

- 2.1 P and NP Informally
- 2.2 Models of Computation
- 2.3 Turing Machines
- 2.4 The Classes P und NP
- 2.5 Nondeterminism = Verification
- 2.6 Karp-Reductions und NP-Completeness
- 2.7 Example of an NP-completeness proof
- 2.8 Important NP-Complete Problems
- 2.9 Optimization Problems

2.1 P and NP Informally

Hard problems

- Some algorithmic problems are “hard nuts” to crack.

- e. g., the *Traveling Salesperson Problem* (TSP):

Given: n cities $S_1, \dots, S_n$,

all $n(n-1)$ pairwise

distances $d(S_i, S_j) \in \mathbb{N} \quad (i \neq j)$



Goal: Shortest round trip through all cities

always exact, always correct polytime

- no general, efficient algorithm known!
(despite decades of intensive research ...)



permutation

$S_{i_1}, S_{i_2}, \dots, S_{i_n}$

$\{1, \dots, n\}$

$\{i_1, \dots, i_n\} = \{1, \dots, n\}$

$$\text{cost}(j) = \sum_{j=1}^{n-1} d(S_{i_j}, S_{i_{j+1}}) + d(S_{i_n}, S_{i_1})$$

Hard problems

- ▶ Some algorithmic problems are “hard nuts” to crack.

- ▶ e. g., the *Traveling Salesperson Problem* (TSP):

Given: n cities $S_1, \dots, S_n$,
all $n(n-1)$ pairwise
distances $d(S_i, S_j) \in \mathbb{N} \quad (i \neq j)$

Goal: Shortest round trip through all cities

always exact, always correct polytime

- ▶ no general, efficient algorithm known!
(despite decades of intensive research ...)

↪ It *seems* as if there is no efficient algorithm for TSP!



Hard problems

- ▶ Some algorithmic problems are “hard nuts” to crack.

- ▶ e. g., the *Traveling Salesperson Problem* (TSP):

Given: n cities $S_1, \dots, S_n$,
all $n(n-1)$ pairwise
distances $d(S_i, S_j) \in \mathbb{N} \quad (i \neq j)$

Goal: Shortest round trip through all cities

always exact, always correct polytime

- ▶ no general, efficient algorithm known!
(despite decades of intensive research ...)

↪ It *seems* as if there is no efficient algorithm for TSP!

- ▶ **But:** can we *prove* that? 

- ▶ Despite similarly intensive research: **No!** (not yet)



Doesn't sound like a shining example for theoretical computer science? ... stay tuned!

United in incapacity



"I can't find an efficient algorithm, but neither can all these famous people."

Garey, Johnson 1979

Complexity Theory

- *Complexity theory* allows us to *compare* the *hardness* of algorithmic problems.



A: old problem
Consensus: hard



B: new problem
Status: unknown
(seems hard to *us* ...)

Complexity Theory

- Complexity theory allows us to *compare* the *hardness* of algorithmic problems.



A: old problem
Consensus: hard

$\leq_p$



B: new problem
Status: unknown
(seems hard to *us* ...)

Intuitive idea:

1. If *A* is a known hard nut, and
2. *B* is at least as hard as *A*,

then *B* is a hard nut, too!

Complexity Theory

- Complexity theory allows us to *compare* the *hardness* of algorithmic problems.

"reduce A to B "



A : old problem
Consensus: hard

$\leq_p$



B : new problem
Status: unknown
(seems hard to *us* ...)

Intuitive idea:

1. If A is a known hard nut, and
2. B is at least as hard as A ,

then B is a hard nut, too!

Formally:

efficient = polytime

1. A is NP-hard: probably $\nexists$ eff. alg. for A
2. $A \leq_p B$: $\exists$ eff. alg. for $B \implies \exists$ eff. alg. for A
 $\rightsquigarrow B$ is NP-hard: probably $\nexists$ eff. alg. for B !

P and NP – Intuitive Synopsis

P = NP?

P and NP – Intuitive Synopsis

- ▶ P = class of problems for which there is an algorithm A and a polynomial p such that A **solves** every instance I in time $O(p(|I|))$.
 - ▶ P for “polynomial” – i. e., all problems where a solution can be *found* by a (deterministic) algorithm in polynomial time.

$P = NP?$

P and NP – Intuitive Synopsis

- ▶ P = class of problems for which there is an algorithm A and a polynomial p such that A **solves** every instance I in time $O(p(|I|))$.
 - ▶ P for “polynomial” – i. e., all problems where a solution can be *found* by a (deterministic) algorithm in polynomial time.
- ▶ NP = class of problems for which there is an algorithm A and a polynomial p such that A can **verify** a given candidate solution $l(I)$ of a given instance I in time $O(p(|I|))$, i. e., check whether $l(I)$ solves I or not.
 - ▶ NP for “nondeterministically polynomial” – i. e., all problems where a solution can be *found* by a *nondeterministic* algorithm in polynomial time.
 - ▶ This is equivalent to the above characterization via verification.

$P = NP?$

P and NP – Intuitive Synopsis

- ▶ P = class of problems for which there is an algorithm A and a polynomial p such that A **solves** every instance I in time $O(p(|I|))$.
 - ▶ P for “polynomial” – i. e., all problems where a solution can be *found* by a (deterministic) algorithm in polynomial time.
- ▶ NP = class of problems for which there is an algorithm A and a polynomial p such that A can **verify** a given candidate solution $l(I)$ of a given instance I in time $O(p(|I|))$, i. e., check whether $l(I)$ solves I or not.
 - ▶ NP for “nondeterministically polynomial” – i. e., all problems where a solution can be *found* by a *nondeterministic* algorithm in polynomial time.
 - ▶ This is equivalent to the above characterization via verification.
- ▶ We know $P \subseteq NP$. We *think* $P \subsetneq NP$, i. e., $P \neq NP$.
The question “ $P = NP$?” is one of the famous **millenium problems** and arguably the **most important open problem of theoretical computer science**.

2.2 Models of Computation

Clicker Question



What is the cost of *adding* two (decimal) d -digit integers?
(For example, for $d = 5$, what is $45\,235 + 91\,342$?)

- A** constant time
- B** logarithmic in d
- C** proportional to d
- D** quadratic in d
- E** no idea what you are talking about



→ sli.do/cs627

Clicker Question



What is the cost of *adding* two (decimal) d -digit integers?
(For example, for $d = 5$, what is $45\,235 + 91\,342$?)

- ☒ **A** constant time ✓
- ☐ **B** ~~logarithmic in d~~
- ☒ **C** proportional to d ✓
- ☐ **D** ~~quadratic in d~~
- ☒ **E** no idea what you are talking about ✓



→ sli.do/cs627

Mathematical Models of Computation

- ▶ complexity classes talk about sets of problems based upon whether they allow an algorithm of a certain cost
 - ▶ in general, this depends on the allowable algorithms and their costs!
- ↪ need to fix a machine model

Mathematical Models of Computation

- ▶ complexity classes talk about sets of problems based upon whether they allow an algorithm of a certain cost
 - ▶ in general, this depends on the allowable algorithms and their costs!
- ↪ need to fix a machine model

A *machine model* decides

- ▶ what algorithms are possible
- ▶ how they are described (= programming language)
- ▶ what an execution *costs*

Goal: Machine models should be
detailed and powerful enough to reflect actual machines,
abstract enough to unify architectures,
simple enough to analyze.

Random Access Machines

Standard model for detailed complexity analysis:

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures*
by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- ▶ unlimited *memory* $\text{MEM}[0], \text{MEM}[1], \text{MEM}[2], \dots$
- ▶ fixed number of *registers* $R_1, \dots, R_r$ (say $r = 100$)

Random Access Machines

Standard model for detailed complexity analysis:

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures*
by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- ▶ unlimited *memory* $\text{MEM}[0], \text{MEM}[1], \text{MEM}[2], \dots$
- ▶ fixed number of *registers* $R_1, \dots, R_r$ (say $r = 100$)
- ▶ memory cells $\text{MEM}[i]$ and registers R_i store w -bit integers, i. e., numbers in $[0..2^w - 1]$
 w is the word width/size; typically $w \propto \lg n \rightsquigarrow 2^w \approx n$

Random Access Machines

Standard model for detailed complexity analysis:

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures*
by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- ▶ unlimited *memory* $\text{MEM}[0], \text{MEM}[1], \text{MEM}[2], \dots$
- ▶ fixed number of *registers* $R_1, \dots, R_r$ (say $r = 100$)
- ▶ memory cells $\text{MEM}[i]$ and registers R_i store w -bit integers, i. e., numbers in $[0..2^w - 1]$
 w is the word width/size; typically $w \propto \lg n \implies 2^w \approx n$
- ▶ Instructions:
 - ▶ load & store: $R_i := \text{MEM}[R_j]$ $\text{MEM}[R_j] := R_i$
 - ▶ operations on registers: $R_k := R_i + R_j$ (arithmetic is *modulo* 2^w !)
also $R_i - R_j$, $R_i \cdot R_j$, $R_i \text{ div } R_j$, $R_i \bmod R_j$
C-style operations (bitwise and/or/xor, left/right shift)
- ▶ conditional and unconditional jumps
- ▶ time cost: number of executed instructions
- ▶ space cost: total number of touched memory cells

RAM-Program Example

Example RAM program

```
1 // Assume:  $R_1$  stores number  $N$ 
2 // Assume:  $\text{MEM}[0..N)$  contains list of  $N$  numbers
3  $R_2 := R_1$ ;
4  $R_3 := R_1 - 2$ ;
5  $R_4 := \text{MEM}[R_3]$ ;
6  $R_5 := R_3 + 1$ ;
7  $R_6 := \text{MEM}[R_5]$ ;
8 if ( $R_4 \leq R_6$ ) goto line 11;
9  $\text{MEM}[R_3] := R_6$ ;
10  $\text{MEM}[R_5] := R_4$ ;
11  $R_3 := R_3 - 1$ ;
12 if ( $R_3 \geq 0$ ) goto line 5;
13  $R_2 := R_2 - 1$ ;
14 if ( $R_2 > 0$ ) goto line 4;
15 // Done:
```

RAM-Program Example

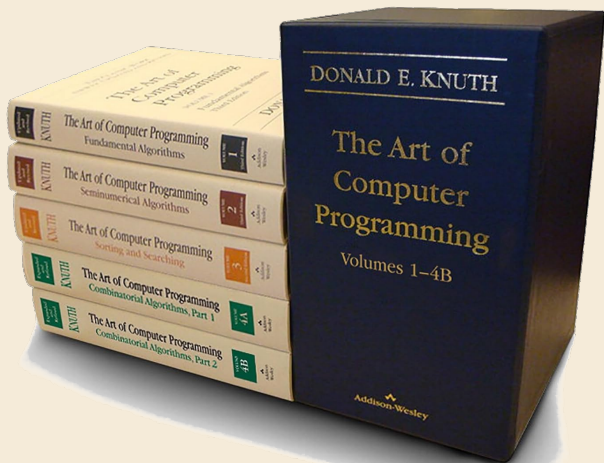
Example RAM program

```
1 // Assume:  $R_1$  stores number  $N$ 
2 // Assume:  $\text{MEM}[0..N)$  contains list of  $N$  numbers
3  $R_2 := R_1$ ;
4  $R_3 := R_1 - 2$ ;
5  $R_4 := \text{MEM}[R_3]$ ;
6  $R_5 := R_3 + 1$ ;
7  $R_6 := \text{MEM}[R_5]$ ;
8 if ( $R_4 \leq R_6$ ) goto line 11;
9  $\text{MEM}[R_3] := R_6$ ;
10  $\text{MEM}[R_5] := R_4$ ;
11  $R_3 := R_3 - 1$ ;
12 if ( $R_3 \geq 0$ ) goto line 5;
13  $R_2 := R_2 - 1$ ;
14 if ( $R_2 > 0$ ) goto line 4;
15 // Done:  $\text{MEM}[0..N)$  sorted
```

RAM-Program Example

Example RAM program

```
1 // Assume:  $R_1$  stores number  $N$ 
2 // Assume:  $\text{MEM}[0..N)$  contains list of  $N$  numbers
3  $R_2 := R_1$ ;
4  $R_3 := R_1 - 2$ ;
5  $R_4 := \text{MEM}[R_3]$ ;
6  $R_5 := R_3 + 1$ ;
7  $R_6 := \text{MEM}[R_5]$ ;
8 if ( $R_4 \leq R_6$ ) goto line 11;
9  $\text{MEM}[R_3] := R_6$ ;
10  $\text{MEM}[R_5] := R_4$ ;
11  $R_3 := R_3 - 1$ ;
12 if ( $R_3 \geq 0$ ) goto line 5;
13  $R_2 := R_2 - 1$ ;
14 if ( $R_2 > 0$ ) goto line 4;
15 // Done:  $\text{MEM}[0..N)$  sorted
```



RAM-Program Example

Example RAM program

```

1 // Assume:  $R_1$  stores number  $N$ 
2 // Assume:  $\text{MEM}[0..N]$  contains list of  $N$  number
3  $R_2 := R_1$ ;
4  $R_3 := R_1 - 2$ ;
5  $R_4 := \text{MEM}[R_3]$ ;
6  $R_5 := R_3 + 1$ ;
7  $R_6 := \text{MEM}[R_5]$ ;
8 if ( $R_4 \leq R_6$ ) goto line 11;
9  $\text{MEM}[R_3] := R_6$ ;
10  $\text{MEM}[R_5] := R_4$ ;
11  $R_3 := R_3 - 1$ ;
12 if ( $R_3 \geq 0$ ) goto line 5;
13  $R_2 := R_2 - 1$ ;
14 if ( $R_2 > 0$ ) goto line 4;
15 // Done:  $\text{MEM}[0..N]$  sorted

```

they need not be examined on subsequent passes. Horizontal lines in Fig. 14 show the progress of the sorting from this standpoint; notice, for example, that five more elements are known to be in final position as a result of Pass 4. On the final pass, no exchanges are performed at all. With these observations we are ready to formulate the algorithm.

Algorithm B (*Bubble sort*). Records $R_1, \dots, R_N$ are rearranged in place; after sorting is complete their keys will be in order, $K_1 \leq \dots \leq K_N$.

- B1. [Initialize BOUND.] Set $\text{BOUND} \leftarrow N$. (BOUND is the highest index for which the record is not known to be in its final position; thus we are indicating that nothing is known at this point.)
- B2. [Loop on j .] Set $t \leftarrow 0$. Perform step B3 for $j = 1, 2, \dots, \text{BOUND} - 1$, and then go to step B4. (If $\text{BOUND} = 1$, this means go directly to B4.)
- B3. [Compare/exchange $R_j : R_{j+1}$.] If $K_j > K_{j+1}$, interchange $R_j \leftrightarrow R_{j+1}$ and set $t \leftarrow j$.
- B4. [Any exchanges?] If $t = 0$, terminate the algorithm. Otherwise set $\text{BOUND} \leftarrow t$ and return to step B2. ■

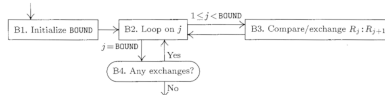


Fig. 15. Flow chart for bubble sorting.

Program B (*Bubble sort*). As in previous MIX programs of this chapter, we assume that the items to be sorted are in locations $\text{INPUT}+1$ through $\text{INPUT}+N$. $\text{r11} \equiv t$; $\text{r12} \equiv j$.

01	START	ENT1 N	1	B1. Initialize BOUND.	$t \leftarrow N$.
02	1H	ST1 BOUND(1:2)	A	BOUND $\leftarrow t$.	
03		ENT2 1	A	B2. Loop on j .	$j \leftarrow 1$.
04		ENT1 0	A	$t \leftarrow 0$.	
05		JMP BOUND	A	Exit if $j \geq \text{BOUND}$.	
06	3H	LDA INPUT,2	C	B3. Compare/exchange $R_j : R_{j+1}$.	
07		CMPA INPUT+1,2	C		
08		JLE 2F	C	No exchange if $K_j \leq K_{j+1}$.	
09		LDX INPUT+1,2	B	R_{j+1}	
10		STX INPUT,2	B	$\rightarrow R_j$.	
11		STA INPUT+1,2	B	(old R_j) $\rightarrow R_{j+1}$.	
12		ENT1 0,2	B	$t \leftarrow j$.	
13	2H	INC2 1	C	$j \leftarrow j + 1$.	
14	BOUND	ENTX $\leftarrow$,2	A + C	$\text{rX} \leftarrow j - \text{BOUND}$.	[Instruction modified]
15		JXN 3B	A + C	Do step B3 for $1 \leq j < \text{BOUND}$.	
16	4H	J1P 1B	A	B4. Any exchanges? To B2 if $t > 0$.	■

2.3 Turing Machines

Keep it Simple, Stupid

- ▶ word-RAM (rather) realistic, but complicated
 - ▶ note that the machine has to grow with the inputs(!)
- ▶ for a coarse distinction of running time complexity, simpler models suffice
 - ▶ useful to reason about “all algorithms”
 - ▶ machine is fixed for all inputs sizes apart from storage for input

Keep it Simple, Stupid

- ▶ word-RAM (rather) realistic, but complicated
 - ▶ note that the machine has to grow with the inputs(!)
- ▶ for a coarse distinction of running time complexity, simpler models suffice
 - ▶ useful to reason about “all algorithms”
 - ▶ machine is fixed for all inputs sizes apart from storage for input

Many models of computation ...

- ▶ ^{word RAM} μ -recursive function
- ▶ Turing machines (TM)
- ▶ counter machines
- ▶ λ -calculus
- ▶ While-programs
- ▶ any Turing-complete language
- ▶ quantum computers
- ▶ ...

... with strong equivalences:

1. all proven to lead to the *same* set of computable functions
2. **Church-Turing thesis:**
any formalization of “effectively computable” is equivalent in this sense
3. **Extended Church-Turing Thesis:**
... and can be simulated with *polynomial overhead* on a TM
 - ▶ true for all on left ...
 - ▶ except theoretical quantum computers!
 - ▶ ignore them for now  wake me when they exist

Turing Machines

- ▶ invented by *Alan Turing* in 1936 as formalization for “computable by hand”

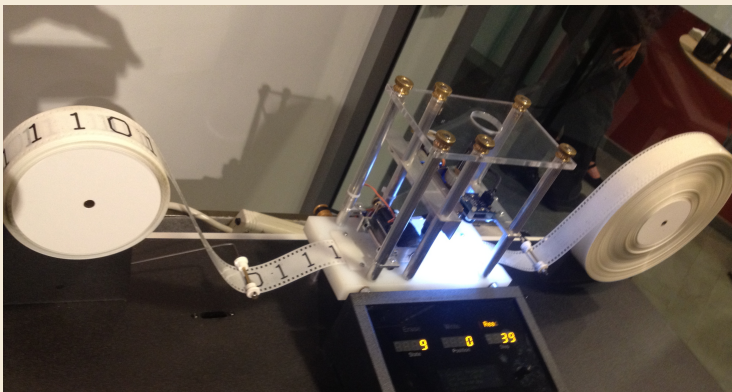
In same paper, Turing proved undecidability of halting problem!

- ▶ *minimalistic* model of universal computer, but can be built:

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO
THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]



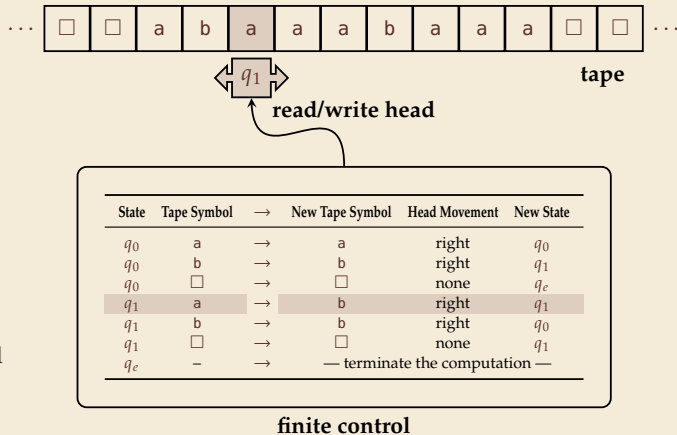
Turingmaschinen in Circulation



Turing Machines – Informal Recap

A *Turing machine* has

- ▶ a **finite control** via states
- ▶ an **input/output-tape**
 - ▶ unbounded length
 - ▶ initially contains input
 - ▶ all other cells contain “□”
- ▶ a **read/write head**
 - ▶ reads the current symbol
 - ▶ overwrites it with a new symbol
 - ▶ initially placed on beginning of input



Turing Machines – Formal Syntax

Definition 2.1 (Turing Machine (TM))

A *Turing machine* is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, q_{\text{halt}})$ with

- ▶ a finite set of *states* Q ,
- ▶ an *input alphabet* Σ ,
- ▶ a *tape alphabet* $\Gamma \supset \Sigma$,
- ▶ for deterministic TMs a *transition function* $\delta : (Q \setminus \{q_{\text{halt}}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, N\}$
for **nondeterministic** TMs a *transition relation* $\delta : (Q \setminus \{q_{\text{halt}}\}) \times \Gamma \rightarrow \underline{2^{Q \times \Gamma \times \{L, R, N\}}}$
- ▶ an *initial state* $q_0 \in Q$,
- ▶ a *blank symbol* $\square \in \Gamma \setminus \Sigma$, and
- ▶ a halting state $q_{\text{halt}} \in Q$

$$2^S = \{s' : s' \subseteq S\}$$

$$\begin{matrix} \text{P} \\ \text{P}(S) \end{matrix} \quad \text{P}(S)$$

Turing Machine – Computation Step

- ▶ Each step of a computation of TM M has the form

$$\delta(q, a) = (q', b, d) \text{ resp. } \delta(q, a) \ni (q', b, d),$$

with the semantics that

- ▶ M is in state $q \neq q_{\text{halt}}$
- ▶ the cell below the read/write head currently contains symbol a
- ▶ M now changes (based on its finite control)
 - ▶ into state q' ,
 - ▶ writes b into the cell under the read/write head
 - ▶ and finally moves the read/write head in direction $d \in \{L, R, N\}$.
($L = \text{left}$, $R = \text{right}$, $N = \text{none (stay)}$)
- ▶ for deterministic TM M , q and a uniquely determine this action;
for nondeterministic TM, we may have several possible actions.
- ▶ to formally define an entire computation, we have to encode the tape contents as well

Turing Machines – Configurations

Definition 2.2 (TM Configuration)

A *configuration (config)* of a TM M is a string $C \in \Gamma^* Q \Gamma^*$.

The semantics of a config $C = \alpha \overset{\hookrightarrow}{q} \beta$, $q \in Q$, is tape content $\alpha\beta$ and head at first symbol of β .

Turing Machines – Configurations

Definition 2.2 (TM Configuration)

A *configuration (config)* of a TM M is a string $C \in \Gamma^* Q \Gamma^*$.

The semantics of a config $C = \alpha q \gamma$, $q \in Q$, is tape content $\alpha\beta$ and head at first symbol of β .

Definition 2.3 (TM Computation Relation)

The *computation relation* $\vdash$ is defined on the set of configurations of a TM M as follows.

$$a_1 \dots a_m q b_1 \dots b_n \vdash \begin{cases} a_1 \dots a_m q' c b_2 \dots b_n, & \delta(q, b_1) = (q', c, N), m \geq 0, n \geq 1, \\ a_1 \dots a_m c q' b_2 \dots b_n, & \delta(q, b_1) = (q', c, R), m \geq 0, n \geq 2, \\ a_1 \dots a_{m-1} \underset{\uparrow}{q'} a_m c b_2 \dots b_n, & \delta(q, b_1) = (q', c, L), m \geq 1, n \geq 1. \end{cases}$$

no movement

Turing Machines – Configurations

Definition 2.2 (TM Configuration)

A *configuration (config)* of a TM M is a string $C \in \Gamma^* Q \Gamma^*$. ◀

The semantics of a config $C = \alpha q \gamma$, $q \in Q$, is tape content $\alpha\beta$ and head at first symbol of β .

Definition 2.3 (TM Computation Relation)

The *computation relation* $\vdash$ is defined on the set of configurations of a TM M as follows.

$$a_1 \dots a_m q b_1 \dots b_n \vdash \begin{cases} a_1 \dots a_m q' c b_2 \dots b_n, & \delta(q, b_1) = (q', c, N), m \geq 0, n \geq 1, \\ a_1 \dots a_m c q' b_2 \dots b_n, & \delta(q, b_1) = (q', c, R), m \geq 0, n \geq 2, \\ a_1 \dots a_{m-1} q' a_m c b_2 \dots b_n, & \delta(q, b_1) = (q', c, L), m \geq 1, n \geq 1. \end{cases}$$

For the boundary case $n = 1$ and direction right, we set

$$a_1 \dots a_m q b_1 \vdash a_1 \dots a_m c q' \square \quad \text{if } \delta(q, b_1) = (q', c, R),$$

For $m = 0$ and direction left, we similarly have set

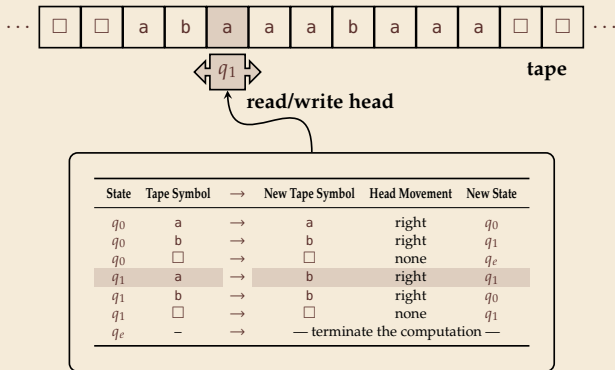
$$q b_1 \dots b_n \vdash q' \square c b_2 \dots b_n \quad \text{if } \delta(q, b_1) = (q', c, L).$$

Turing Machines – Configuration Example

Turing Machines – Configuration Example

Example:

For the shown TM M ,
the current configuration is:
 $C = ab q_1 aaabaaa$



- ▶ TM Config $C = \alpha q \beta$ completely describes current state of computation
 - ▶ $\alpha \beta$ is the (non-blank) tape content
 - ▶ q is the current state of the TM
 - ▶ the read/write head is on the first symbol of β

Turing Machines – Computed Function

- ▶ With this setup, we can now formally define what a Turing machine computes.

Definition 2.4 (Function computed by a TM)

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, q_{\text{halt}})$ be a TM. The function computed by M on input $x \in \Sigma^*$, written $M(x)$, is defined as

$$M(x) = \{y : q_0 x \vdash^* q_{\text{halt}} y\}.$$

For deterministic TMs, we will also have $|M(x)| \leq 1$ and we write $M(x) = y$ for $M(x) = \{y\}$. ◀

Turing Machines – Computed Function

- With this setup, we can now formally define what a Turing machine computes.

Definition 2.4 (Function computed by a TM)

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, q_{\text{halt}})$ be a TM. The function computed by M on input $x \in \Sigma^*$, written $M(x)$, is defined as

$$M(x) = \{y : q_0x \vdash^* q_{\text{halt}}y\}.$$

For deterministic TMs, we will also have $|M(x)| \leq 1$ and we write $M(x) = y$ for $M(x) = \{y\}$. ◀

Definition 2.5 (Time and Space cost)

For a TM M and input $x \in \Sigma^*$, we define

$$\text{time}_M(x) = \inf\{t : q_0x \vdash^{\textcircled{t}} q_{\text{halt}}y\} \cup \{0\}.$$

We define $\text{space}_M(x) = \inf\{|\alpha\beta| : q_0x \vdash^* \alpha q\beta \vdash^* q_{\text{halt}}y, |\alpha\beta|_{\square} \leq 2\} \cup \{0\}$. ◀

not important

$\square$ in $\alpha\beta$

- Note: *time* and *space* can be ∞ or 0 for nondeterministic TMs.

Turing Machines – Accepted Language

► Often convenient to use language acceptance instead of function computation.

► for deterministic TM, compute *characteristic function* of L : $\mathbb{1}_L(x) = \begin{cases} 1 & \text{if } x \in L \\ 0 & \text{otherwise} \end{cases}$

► care needed for nondeterministic TM

Turing Machines – Accepted Language

- ▶ Often convenient to use language acceptance instead of function computation.
- ▶ for deterministic TM, compute *characteristic function* of L :
$$\mathbb{1}_L(x) = \begin{cases} 1 & \text{if } x \in L \\ 0 & \text{otherwise} \end{cases}$$
- ▶ care needed for nondeterministic TM

Definition 2.6 (Language of TM)

The language $\mathcal{L}(M)$ accepted by a TM M is defined as

$$\mathcal{L}(M) = \{w \in \Sigma^* : \underbrace{1 \in M(w)}\}.$$

$\rightsquigarrow$

nondeterministic TM accepts w iff some computation accepts w

Turing Machines – Several tapes

Remark 2.7 (k -tape TMs)

We only consider one-tape TMs here. In general, k -tape TMs can be faster.

However, any language accepted by a k -tape TM in time $f(n)$ is also accepted by a 1-tape TM with running time $O(f^2(n))$.

The models are thus polynomially equivalent.



Turing Machines – Totality

- In complexity theory, we will restrict ourselves to TMs that always halt.

Definition 2.8 (Terminating TM)

A TM M is *always terminating / total* if there is a function $T : \mathbb{N} \rightarrow \mathbb{N}$ such that $q_0x = C_0 \vdash C_1 \vdash \dots \vdash C_t$ implies $t \leq T(|x|)$, and there is a y such that $q_0x \vdash^* q_{\text{halt}}y$. ◀

- Note that in a terminating TM, we always have $1 \leq \text{time}_M(x) \leq T(|x|)$.

Turing Machines – Totality

- In complexity theory, we will restrict ourselves to TMs that always halt.

Definition 2.8 (Terminating TM)

A TM M is *always terminating / total* if there is a function $T : \mathbb{N} \rightarrow \mathbb{N}$ such that $q_0x = C_0 \vdash C_1 \vdash \dots \vdash C_t$ implies $t \leq T(|x|)$, and there is a y such that $q_0x \vdash^* q_{\text{halt}}y$. ◀

- Note that in a terminating TM, we always have $1 \leq \text{time}_M(x) \leq T(|x|)$.

Lemma 2.9 (Time-constrained TM)

Given a (potentially nonterminating) TM M and a function $T : \mathbb{N} \rightarrow \mathbb{N}$ **computable** in time $T(n)$, we can construct an *always terminating* TM M' that simulates M for $T(|x|)$ on x and outputs TIMEOUT if M has not terminated yet. Moreover, $\text{time}_{M'}(x) = O(T^2(|x|))$ for all $x \in \Sigma^*$. ◀

Idea: M' computes $T(|x|)$ and write $T(|x|)$ many $\textcircled{2} \in \Gamma$
then simulate M step by step, then delete $\textcircled{2}$
when no $\textcircled{2}$ left, output TIMEOUT

Turing Machines – Totality

- In complexity theory, we will restrict ourselves to TMs that always halt.

Definition 2.8 (Terminating TM)

A TM M is *always terminating / total* if there is a function $T : \mathbb{N} \rightarrow \mathbb{N}$ such that $q_0x = C_0 \vdash C_1 \vdash \dots \vdash C_t$ implies $t \leq T(|x|)$, and there is a y such that $q_0x \vdash^* q_{\text{halt}}y$. ◀

- Note that in a terminating TM, we always have $1 \leq \text{time}_M(x) \leq T(|x|)$.

Lemma 2.9 (Time-constrained TM)

Given a (potentially nonterminating) TM M and a function $T : \mathbb{N} \rightarrow \mathbb{N}$ **computable** in time $T(n)$, we can construct an *always terminating* TM M' that simulates M for $T(|x|)$ on x and outputs TIMEOUT if M has not terminated yet. Moreover, $\text{time}_{M'}(x) = O(T^2(|x|))$ for all $x \in \Sigma^*$. ◀



If we are only interested in the (non)existence of a polynomial-time TM, we can restrict ourselves to total TMs that will never TIMEOUT.

Models of Computation – Summary

- ▶ Concrete model such as TMs useful for some proofs
- ▶ Often, details do not matter as long as models are polynomially equivalent
 - ▶ Note: TM always means we are in the logarithmic cost model for arithmetic operations $\omega = \mathcal{O}(\log)$
- ▶ In the following, discuss more abstract notion of “algorithm”
(fine to substitute by TM in each case)

2.4 The Classes P und NP

Worst Case Complexity

Definition 2.10 (Time and Space Complexity – Generic)

Let Σ_I and Σ_O two alphabets and A an algorithm implementing a **total** mapping $\Sigma_I^* \rightarrow \Sigma_O^*$. Then for each $x \in \Sigma_I^*$ we denote by $time_A(x)$ (resp. $space_A(x)$) the logarithmic time complexity (resp. logarithmic space complexity) for A on x . ◀

Worst Case Complexity

Definition 2.10 (Time and Space Complexity – Generic)

Let Σ_I and Σ_O two alphabets and A an algorithm implementing a **total** mapping $\Sigma_I^* \rightarrow \Sigma_O^*$. Then for each $x \in \Sigma_I^*$ we denote by $time_A(x)$ (resp. $space_A(x)$) the logarithmic time complexity (resp. logarithmic space complexity) for A on x . ◀

Where needed, we can unpack this in full detail for Turing machines!

Worst Case Complexity

Definition 2.10 (Time and Space Complexity – Generic)

Let Σ_I and Σ_O two alphabets and A an algorithm implementing a **total** mapping $\Sigma_I^* \rightarrow \Sigma_O^*$. Then for each $x \in \Sigma_I^*$ we denote by $time_A(x)$ (resp. $space_A(x)$) the logarithmic time complexity (resp. logarithmic space complexity) for A on x . ◀

Where needed, we can unpack this in full detail for Turing machines!

Definition 2.11 (Worst-Case Complexity)

Let Σ_I and Σ_O be two alphabets and A an algorithm implementing a **total** mapping $\Sigma_I^* \rightarrow \Sigma_O^*$. The *worst case time complexity of A* is the function $Time_A : \mathbb{N} \rightarrow \mathbb{N}$ with

$$Time_A(n) = \max\{time_A(x) : x \in \Sigma_I^{\circledast n}\},$$

for each $n \in \mathbb{N}$. The *worst case space complexity of A* is given by function $Space_A : \mathbb{N} \rightarrow \mathbb{N}$ with

$$Space_A(n) = \max\{space_A(x) : x \in \Sigma_I^n\}. \quad \blacktriangleleft$$

Decision Problems = Languages

Definition 2.12 (Decision Problem and Algorithms)

A *decision problem* is given by $P = (L, U, \Sigma)$ for Σ an alphabet and $L \subseteq U \subseteq \Sigma^*$. An algorithm A *solves (decides)* decision problem P , if for all $x \in U$

1. $A(x) = 1$ for $x \in L$, and
2. $A(x) = 0$ for $x \in U \setminus L$ (i. e., $x \notin L$)

holds. Here $A(x)$ denotes the output of A on input x .

If $U = \Sigma^*$ holds we denote P briefly by (L, Σ) . 

$\rightsquigarrow A$ computes a total function $A : U \rightarrow \{0, 1\}$,
the *characteristic function* $\mathbb{1}_L : U \rightarrow \{0, 1\}$ of language L .
We then write $L = \mathcal{L}(A)$, the language accepted by A .

Decision Problems = Languages

Definition 2.12 (Decision Problem and Algorithms)

A *decision problem* is given by $P = (L, U, \Sigma)$ for Σ an alphabet and $L \subseteq U \subseteq \Sigma^*$. An algorithm A *solves (decides)* decision problem P , if for all $x \in U$

1. $A(x) = 1$ for $x \in L$, and
2. $A(x) = 0$ for $x \in U \setminus L$ (i. e., $x \notin L$)

holds. Here $A(x)$ denotes the output of A on input x .

If $U = \Sigma^*$ holds we denote P briefly by (L, Σ) . 

$\rightsquigarrow A$ computes a total function $A : U \rightarrow \{0, 1\}$,
the *characteristic function* $\mathbb{1}_L : U \rightarrow \{0, 1\}$ of language L .
We then write $L = \mathcal{L}(A)$, the language accepted by A .

We restrict our attention
to *decision problems*:

Given: $w \in \Sigma^*$.

Goal: Is $w \in L$?

Example:

w is an encoding of an instance of the
traveling salesperson problem **and** a threshold D

$L = \{w : w \text{ encodes instance } w / \text{ opt. round trip length } \leq D\}$

Optimal Algorithms

Definition 2.13 (Upper/Lower Bounds, Optimal Algorithms)

Let U be an algorithmic problem and f, g functions $\mathbb{N}_0 \rightarrow \mathbb{R}^+$.

- ▶ We call $O(g(n))$ an upper bound for time complexity of U if there is an algorithm A that solves U in time $Time_A(n) \in O(g(n))$.
- ▶ We say $\Omega(f(n))$ is a lower bound for time complexity of U if every algorithm A that solves U needs time $Time_A(n) \in \Omega(f(n))$.
- ▶ An algorithm A is called optimal for U if $Time_A(n) \in O(g(n))$ and $\Omega(g(n))$ is a lower bound for the time complexity of U .



Running Time

Definition 2.14 (time classes)

For function $f : \mathbb{N} \rightarrow \mathbb{N}$, the class $TIME(f(n))$ is the set of all languages A , for which there is a *deterministic* Turing machine M with $\mathcal{L}(M) = A$ and $time_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$. ◀

Running Time

Definition 2.14 (time classes)

For function $f : \mathbb{N} \rightarrow \mathbb{N}$, the class $TIME(f(n))$ is the set of all languages A , for which there is a *deterministic* Turing machine M with $\mathcal{L}(M) = A$ and $time_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$. ◀

Definition 2.15 (P, tractable)

We define the class of languages P decidable in polynomial time by

$$P := \bigcup_{p \text{ polynomial}} TIME(p(n)). = \bigcup_{c \in \mathbb{N}} TIME(n^c)$$

A language (a decision problem) $L \in P$ is called *tractable / efficiently decidable*. ◀

Nondeterministic Running Time

Recall:

- ▶ A nondeterministic Turing machine / algorithm M accepts L ($\mathcal{L}(M) = L$) if for all $x \in L$ there is at least one computation of M which accepts x and for all $y \notin L$ every computation of M rejects y .
- ▶ We only consider always-terminating Turing machines.
- ▶ The running time $time_M(x)$ of M on x is given by the longest computation of M on x .

Nondeterministic Running Time

Recall:

- ▶ A nondeterministic Turing machine / algorithm M accepts L ($\mathcal{L}(M) = L$) if for all $x \in L$ there is at least one computation of M which accepts x and for all $y \notin L$ every computation of M rejects y .
- ▶ We only consider always-terminating Turing machines.
- ▶ The running time $time_M(x)$ of M on x is given by the longest computation of M on x .

Definition 2.16 (NTIME, NP)

For function $f : \mathbb{N} \rightarrow \mathbb{N}$, the class $\underline{NTIME}(f(n))$ is the set of all languages A , for which there is a nondeterministic Turing machine M with $\mathcal{L}(M) = A$ and $time_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$.

Nondeterministic Running Time

Recall:

- ▶ A nondeterministic Turing machine / algorithm M accepts L ($\mathcal{L}(M) = L$) if for all $x \in L$ there is at least one computation of M which accepts x and for all $y \notin L$ every computation of M rejects y .
- ▶ We only consider always-terminating Turing machines.
- ▶ The running time $time_M(x)$ of M on x is given by the longest computation of M on x .

Definition 2.16 (NTIME, NP)

For function $f : \mathbb{N} \rightarrow \mathbb{N}$, the class $NTIME(f(n))$ is the set of all languages A , for which there is a **nondeterministic** Turing machine M with $\mathcal{L}(M) = A$ and $time_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$.

The class of languages **NP** is defined by

$$\text{NP} := \bigcup_{p \text{ polynomial}} NTIME(p(n)).$$



2.5 Nondeterminism = Verification

Nondeterminism?

- ▶ The original definition of **NP** via nondeterministic Turing machines is not very intuitive.
- ▶ There is an equivalent characterization that is usually more convenient to use:
Certificates and verifiers.

Polynomially verifiable

Definition 2.17 (Certificates, Verifier, VP)

Let $L \subseteq \Sigma^*$ be a language.

- ▶ An algorithm A acting on inputs from $\Sigma^* \times \{0, 1\}^*$ is called *verifier for L* (notation $L = \mathcal{V}(A)$), if

$$L = \{w \in \Sigma^* : \exists c \in \{0, 1\}^* A(w, c) = 1\}.$$

If A accepts input (w, c) we say c is *proof* or *certificate* for $w \in L$.

- ▶ A verifier A for L is a *polynomial-time verifier* if there is a $d \in \mathbb{N}$ such that for all $w \in L$, there is a proof c (for $w \in L$) with $\text{time}_A(w, c) \in O(|w|^d)$.
- ▶ We define the class of polynomially verifiable languages **VP** by

$$\mathbf{VP} = \{\mathcal{V}(A) : A \text{ is polynomial time verifier}\}.$$

$$\exists \quad L = \{ \text{TSP Yes instances} \mid d, \mathbb{D} \}$$

Nondeterminism $\leftrightarrow$ certificate

Theorem 2.18

NP = VP.

Proof, $\bullet$ NP $\subseteq$ VP

$$\exists c \text{ Time}_M(n) = O(n^c)$$

$L \in \text{NP} \leadsto \exists$ nondet. TM $M : \mathcal{Q}(M) = L$, polytime

Construct verifier A , $L \subseteq \Sigma^*$

$$(x, c) \in \Sigma^* \times \{0,1\}^*$$

A uses c as guide to pick nondeterministic computation

at each point in time, M has $|\mathcal{S}(q, a)| \leq G$ options

use $\lceil \log G \rceil$ bits in c to select one step

M accepts $x \iff \exists$ accepting computation $(q_0 x \vdash^* q_{\text{halt}} 1)$ for x

$\iff \exists c$ that guides A to accept (x, c)

$$\Rightarrow V(A) = L \Rightarrow L \in \text{VP}$$

• $VP \subseteq NP$

start with polynomial verifier A for $L \in VP$

construct nondet. TM M

- ① M 'guesses' nondeterministically binary string c
- ② simulate A

$\mathcal{L}(M) = L \Rightarrow L \in NP$

□

$TSP \in NP = VP$

check length of given tour $\in D$

easy ☺

2.6 Karp-Reductions und NP-Completeness

Recap



We restrict our attention
to *decision problems*:

Given: $w \in \Sigma^*$.

Goal: Is $w \in L$?

Example:

w is an encoding of an instance of the
traveling salesperson problem **and** a threshold D

$$L = \{w : w \text{ encodes instance } w / \text{opt. round trip length} \leq D\}$$

Recap



We restrict our attention to *decision problems*:

Given: $w \in \Sigma^*$.

Goal: Is $w \in L$?

Example:

w is an encoding of an instance of the traveling salesperson problem **and** a threshold D

$$L = \{w : w \text{ encodes instance } w / \text{opt. round trip length} \leq D\}$$

$\rightsquigarrow$ problems = (formal) languages $L \subseteq \Sigma^*$

▶ problem instance = word $w \in \Sigma^*$

▶ $w \in \Sigma^*$ is a *Yes instance* if $w \in L$, otherwise a *No instance*

Recap

▶ We restrict our attention to *decision problems*:

Given: $w \in \Sigma^*$.

Goal: Is $w \in L$?

Example:

w is an encoding of an instance of the traveling salesperson problem **and** a threshold D

$L = \{w : w \text{ encodes instance } w / \text{ opt. round trip length} \leq D\}$

↪ problems = (formal) languages $L \subseteq \Sigma^*$

▶ problem instance = word $w \in \Sigma^*$

▶ $w \in \Sigma^*$ is a *Yes instance* if $w \in L$, otherwise a *No instance*

↪ For problems on structures, e. g., graphs, we need an *encoding* of the instance as a string.
(often simple; standard data structures do the trick)

↪ input size n of instance = length of the *encoding* of instance

↪ all running times are worst case over instances of encoding length n

Karp Reductions



A: old problem
Consensus: hard

$\leq_p$



B: new problem
Status: unknown
(seems hard to *us* ...)

► **Goal:** Show that *B* is at least as hard as *A*.

short for: deterministic TM *M* w/ $\text{Time}_M(n) = O(n^k)$ for constant *k*.



Assume there *was* a polytime algorithm *M* für *B*.

Solve *A* using *M* (in polytime).

⇒ polytime algo for *B* implies polytime algo for *A*

⇒ *B* at least as hard as *A*

Karp Reductions



A: old problem
Consensus: hard

$\leq_p$



B: new problem
Status: unknown
(seems hard to *us* ...)

► **Goal:** Show that B is at least as hard as A .

short for: deterministic TM M w/ $\text{Time}_M(n) = O(n^k)$ for constant k .



Assume there *was* a polytime algorithm M für B .
Solve A using M (in polytime).

$\rightsquigarrow$ polytime algo for B implies polytime algo for A

$\rightsquigarrow B$ at least as hard as A

Formally: (strong notion than intuition above!)

Definition 2.19 (polytime reduction, $\leq_p$)

Let $A \subseteq \Sigma^*$ and $B \subseteq \Gamma^*$ be languages (decision problems).

A is *polytime reducible* to B – written $A \leq_p B$ – if there is a total function $g : \Sigma^* \rightarrow \Gamma^*$, computable in polynomial time, with

$$\forall w \in \Sigma^* : w \in A \Leftrightarrow g(w) \in B.$$

- This type of reduction is called a *Karp-reduction*
- It is more restrictive than our intuitive version would need, but allows finer complexity classification (NP and co-NP)

Implication of Reductions

Lemma 2.20 (Membership reduction)

If $A \leq_p B$ and $B \in P$ (resp. $B \in NP$), then $A \in P$ (resp. $A \in NP$).



Implication of Reductions

Lemma 2.20 (Membership reduction)

If $A \leq_p B$ and $B \in \text{P}$ (resp. $B \in \text{NP}$), then $A \in \text{P}$ (resp. $A \in \text{NP}$).

Proof:

Since $B \in \text{P}$, there is polytime TM M with $\mathcal{L}(M) = B$.

Since $A \leq_p B$, there further is polytime TM g mit $\forall w : w \in A \Leftrightarrow g(w) \in B$ (*).



Implication of Reductions

Lemma 2.20 (Membership reduction)

If $A \leq_p B$ and $B \in \text{P}$ (resp. $B \in \text{NP}$), then $A \in \text{P}$ (resp. $A \in \text{NP}$).

Proof:

Since $B \in \text{P}$, there is polytime TM M with $\mathcal{L}(M) = B$.

Since $A \leq_p B$, there further is polytime TM g mit $\forall w : w \in A \Leftrightarrow g(w) \in B$ (*).

We construct TM M' for A :

first simulate g on input w , then simulate M on $g(w)$.



Implication of Reductions

Lemma 2.20 (Membership reduction)

If $A \leq_p B$ and $B \in \mathbf{P}$ (resp. $B \in \mathbf{NP}$), then $A \in \mathbf{P}$ (resp. $A \in \mathbf{NP}$).

Proof:

Since $B \in \mathbf{P}$, there is polytime TM M with $\mathcal{L}(M) = B$.

Since $A \leq_p B$, there further is polytime TM g mit $\forall w : w \in A \Leftrightarrow g(w) \in B$ (*).

We construct TM M' for A :

first simulate g on input w , then simulate M on $g(w)$.

Since M and g are polytime TMs, so is M' , and $\mathcal{L}(M') = A$ (since (*)).

$\rightsquigarrow A \in \mathbf{P}$.

(The version with $B \in \mathbf{NP}$ is similar, just using nondeterministic polytime).

NP Completeness

Definition 2.21 (NP-hard, NP-complete)

A language A is called NP-*hard*, if we have for **all** languages $L \in \text{NP}$ that $L \leq_p A$.

A language A is called NP-*complete*, if A is NP-hard and $A \in \text{NP}$.



NP Completeness

Definition 2.21 (NP-hard, NP-complete)

A language A is called NP-*hard*, if we have for **all** languages $L \in \text{NP}$ that $L \leq_p A$.

A language A is called NP-*complete*, if A is NP-hard and $A \in \text{NP}$.

Theorem 2.22 (One for all and all for one)

For an NP-complete language A holds: $A \in \text{P} \iff \text{P} = \text{NP}$.

NP Completeness

Definition 2.21 (NP-hard, NP-complete)

A language A is called NP-*hard*, if we have for **all** languages $L \in \text{NP}$ that $L \leq_p A$.

A language A is called NP-*complete*, if A is NP-hard and $A \in \text{NP}$.

Theorem 2.22 (One for all and all for one)

For an NP-complete language A holds: $A \in \text{P} \iff \text{P} = \text{NP}$.

$\rightsquigarrow$ Under the *consensus hypothesis* that $\text{P} \subsetneq \text{NP}$, this means that for an NP-complete problem A , we should expect **no efficient solution** for A .

NP Completeness

Definition 2.21 (NP-hard, NP-complete)

A language A is called NP-*hard*, if we have for **all** languages $L \in \text{NP}$ that $L \leq_p A$.

A language A is called NP-*complete*, if A is NP-hard and $A \in \text{NP}$.

Theorem 2.22 (One for all and all for one)

For an NP-complete language A holds: $A \in \text{P} \iff \text{P} = \text{NP}$.

$\rightsquigarrow$ Under the *consensus hypothesis* that $\text{P} \subsetneq \text{NP}$, this means that for an NP-complete problem A , we should expect **no efficient solution** for A .

Proof:

$\Rightarrow$ Let $L \in \text{NP}$ be arbitrary.

Since A is (by assumption) NP-hard, we have $L \leq_p A$.

Since $A \in \text{P}$, by membership reduction (Lemma 2.20) also $L \in \text{P}$.

Since L was an *arbitrary* language from NP , $\text{P} = \text{NP}$ follows.

NP Completeness

Definition 2.21 (NP-hard, NP-complete)

A language A is called NP-*hard*, if we have for **all** languages $L \in \text{NP}$ that $L \leq_p A$.

A language A is called NP-*complete*, if A is NP-hard and $A \in \text{NP}$.

Theorem 2.22 (One for all and all for one)

For an NP-complete language A holds: $A \in \text{P} \iff \text{P} = \text{NP}$.

$\rightsquigarrow$ Under the *consensus hypothesis* that $\text{P} \subsetneq \text{NP}$, this means that for an NP-complete problem A , we should expect **no efficient solution** for A .

Proof:

$\Rightarrow$ Let $L \in \text{NP}$ be arbitrary.

Since A is (by assumption) NP-hard, we have $L \leq_p A$.

Since $A \in \text{P}$, by membership reduction (Lemma 2.20) also $L \in \text{P}$.

Since L was an *arbitrary* language from NP , $\text{P} = \text{NP}$ follows.

$\Leftarrow$ Let conversely $\text{P} = \text{NP}$.

Then, by assumption, $A \in \text{NP} = \text{P}$.

Implications of NP-Completeness

- ▶ NP-completeness tells us a lot about a problem!



But shall we possible prove $L \leq_p A$ *for all* possible $L \in \text{NP}$?

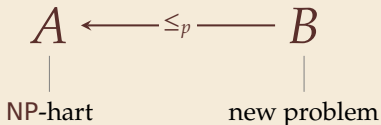
Implications of NP-Completeness

- ▶ NP-completeness tells us a lot about a problem!



But shall we possible prove $L \leq_p A$ *for all* possible $L \in \text{NP}$?

- ▶ One can show: $\leq_p$ is a *transitive* relation on languages.
(proof is similar to membership-reduction lemma)



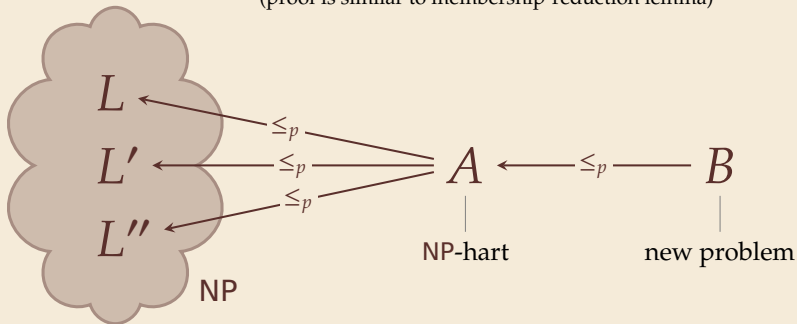
Implications of NP-Completeness

- ▶ NP-completeness tells us a lot about a problem!



But shall we possibly prove $L \leq_p A$ *for all* possible $L \in \text{NP}$?

- ▶ One can show: $\leq_p$ is a *transitive* relation on languages.
(proof is similar to membership-reduction lemma)



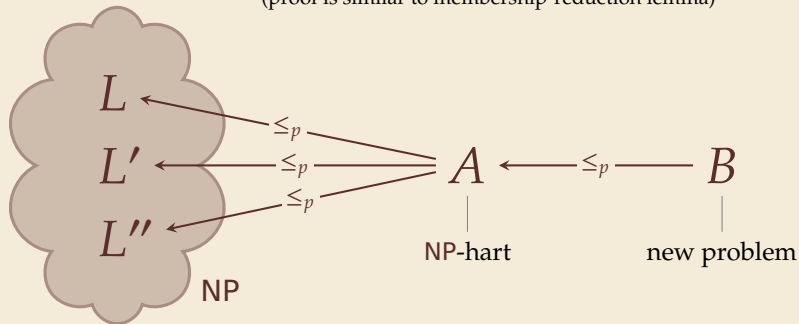
Implications of NP-Completeness

- NP-completeness tells us a lot about a problem!



But shall we possible prove $L \leq_p A$ *for all* possible $L \in \text{NP}$?

- One can show: $\leq_p$ is a *transitive* relation on languages.
(proof is similar to membership-reduction lemma)



A NP-hard and $A \leq_p B$
 $\rightsquigarrow B$ NP-hard

Clicker Question

What is the value of the formula φ under the variable assignment V ?

$$\varphi = x_1 \wedge (\neg x_2 \vee x_3)$$

$$V = \{x_1 \mapsto \text{true}, x_2 \mapsto \text{true}, x_3 \mapsto \text{false}\}$$



- A** satisfiable
- B** *true*
- C** *false*
- D** not defined
- E** No idea.



→ sli.do/cs627

Clicker Question

What is the value of the formula φ under the variable assignment V ?

$$\varphi = x_1 \wedge (\neg x_2 \vee x_3)$$

$$V = \{x_1 \mapsto \text{true}, x_2 \mapsto \text{true}, x_3 \mapsto \text{false}\}$$



A ~~satisfiable~~

D ~~not defined~~

B ~~true~~

E No idea. ✓

C false ✓



→ sli.do/cs627

The Mother of All Problems

- It remains to identify a first NP-complete problem!

Are there any *at all*?

The Mother of All Problems

- It remains to identify a first NP-complete problem!

Are there any *at all*?

Theorem 2.23 (Cook-Levin)

SAT is NP-complete.

- SAT is the satisfiability problem of propositional logic:
Given: Boolean (propositional logic) formula φ over variables $x_1, \dots, x_n$.
Goal: Is there a variable assignment $V : \{x_1, \dots, x_n\} \rightarrow \{true, false\}$,
so that φ evaluates under V to *true*?

The Mother of All Problems

- It remains to identify a first NP-complete problem!

Are there any *at all*?

Theorem 2.23 (Cook-Levin)

SAT is NP-complete.

- SAT is the satisfiability problem of propositional logic:

Given: Boolean (propositional logic) formula φ over variables $x_1, \dots, x_n$.

Goal: Is there a variable assignment $V : \{x_1, \dots, x_n\} \rightarrow \{\text{true}, \text{false}\}$,
so that φ evaluates under V to *true*?

Proof (Theorem 2.23):

Idea: Given any nondeterministic TM M for an arbitrary language $L \in \text{NP}$, construct from a word w a formula $\varphi(w)$, which exactly encodes all valid computations of M .

Variables $x_{q,t}$: Is M be in state q at time t ?

$y_{c,i,t}$: Does tape cell i contain char c at time t ?

$z_{i,t}$ Does read/write head stand at position i at time t ?

$\rightsquigarrow \varphi(w)$ satisfiable iff M accepts w .

(for details, see, e. g., §2.3 of S. Arora and B. Barak: *Computational Complexity: A Modern Approach*)

2.7 Example of an NP-completeness proof

3SAT

- ▶ Let's do a more typical full example.

3SAT

- ▶ Let's do a more typical full example.
- ▶ Need one more NP-complete problem first

Definition 2.24 (3SAT)

Given: A Boolean formula φ in 3-CNF:

conjunctive normal form with *at most* 3 literals per clause

Goal: Is there an assignment V of the variables in φ , so that φ evaluates to *true*?
(a.k.a. Is φ *satisfiable*?)

Example:

$(x_1 \vee \neg x_3 \vee x_2) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (x_5 \vee \underbrace{\neg x_5}_{\text{literal}} \vee \neg x_1) \wedge \underbrace{(x_1 \vee x_3 \vee x_5)}_{\text{clause}}$
satisfiable, e. g., via $x_1 \mapsto \text{true}$, $x_5 \mapsto \text{false}$ (other variables arbitrary)

3SAT

- ▶ Let's do a more typical full example.
- ▶ Need one more NP-complete problem first

Definition 2.24 (3SAT)

Given: A Boolean formula φ in 3-CNF:

*conjunctive normal form with **at most** 3 literals per clause*

Goal: Is there an assignment V of the variables in φ , so that φ evaluates to *true*?
(a.k.a. Is φ *satisfiable*?)

Example:

$(x_1 \vee \neg x_3 \vee x_2) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (x_5 \vee \neg x_5 \vee \neg x_1) \wedge (x_1 \vee x_3 \vee x_5)$

satisfiable, e. g., via $x_1 \mapsto \text{true}$, $x_5 \mapsto \text{false}$ (other variables arbitrary)

Theorem 2.25 (3SAT)

$\text{SAT} \leq_p \text{3SAT}$ and $\text{3SAT} \in \text{NP}$.

Corollary 2.26 (3SAT)

3SAT is NP-complete.

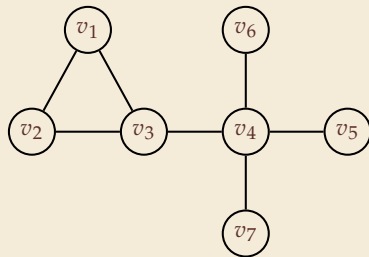
Vertex Cover

Definition 2.27 (VERTEXCOVER)

Given: A (simple, undirected) graph $G = (V, E), E \subseteq \binom{V}{2}$,
threshold k .

Goal: $\exists S \subseteq V$ with $|S| \leq k$, such that $\forall e \in E : S \cap e \neq \emptyset$? ◀

Intuitively: a small subset S of vertices of a graph,
such that every edge is *covered* by S



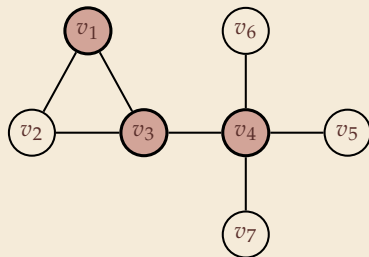
Vertex Cover

Definition 2.27 (VERTEXCOVER)

Given: A (simple, undirected) graph $G = (V, E), E \subseteq \binom{V}{2}$,
threshold k .

Goal: $\exists S \subseteq V$ with $|S| \leq k$, such that $\forall e \in E : S \cap e \neq \emptyset$? ◀

Intuitively: a small subset S of vertices of a graph,
such that every edge is *covered* by S



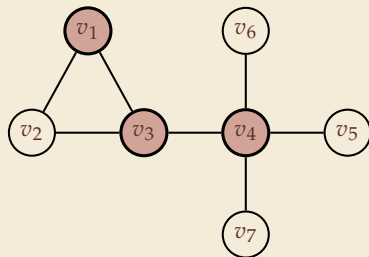
Vertex Cover

Definition 2.27 (VERTEXCOVER)

Given: A (simple, undirected) graph $G = (V, E), E \subseteq \binom{V}{2}$, threshold k .

Goal: $\exists S \subseteq V$ with $|S| \leq k$, such that $\forall e \in E : S \cap e \neq \emptyset$ ◀

Intuitively: a small subset S of vertices of a graph, such that every edge is *covered* by S



Theorem 2.28 (VERTEXCOVER hard)

VERTEXCOVER is NP-complete.

Proof:

We will prove (i) VERTEXCOVER $\in$ VP and (ii) 3SAT $\leq_p$ VERTEXCOVER.

↪ Theorem 2.28 (since 3SAT is NP-complete and VP = NP).



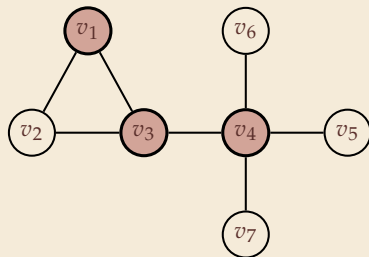
Vertex Cover

Definition 2.27 (VERTEXCOVER)

Given: A (simple, undirected) graph $G = (V, E), E \subseteq \binom{V}{2}$, threshold k .

Goal: $\exists S \subseteq V$ with $|S| \leq k$, such that $\forall e \in E : S \cap e \neq \emptyset$? ◀

Intuitively: a small subset S of vertices of a graph, such that every edge is *covered* by S



Theorem 2.28 (VERTEXCOVER hard)

VERTEXCOVER is NP-complete. ◀

Proof:

We will prove (i) VERTEXCOVER $\in$ VP and (ii) 3SAT $\leq_p$ VERTEXCOVER.

↪ Theorem 2.28 (since 3SAT is NP-complete and VP = NP).

(i) certificate = S ; verifier whether all edges $e \in E$ are covered and whether $|S| \leq k$

↪ clearly doable in polytime ↪ VERTEXCOVER $\in$ VP.

(ii) $3\text{SAT} \leq_p \text{VERTEXCOVER}$

Proof:

- Intuition: Express 3SAT instance as a VERTEXCOVER instance.



(ii) $3\text{SAT} \leq_p \text{VERTEXCOVER}$

Proof:

► Intuition: Express 3SAT instance as a VERTEXCOVER instance.

► So, let φ be an arbitrary formula in 3-CNF over variables $x_1, \dots, x_m$

$\rightsquigarrow \varphi$ has the form $\underbrace{(l_{1,1} \vee l_{1,2} \vee l_{1,3})}_{C_1} \wedge \underbrace{(l_{2,1} \vee l_{2,2} \vee l_{2,3})}_{C_2} \wedge \dots \wedge \underbrace{(l_{n,1} \vee l_{n,2} \vee l_{n,3})}_{C_n},$

with $l_{i,j} \in \{x_1, \neg x_1, \dots, x_m, \neg x_m\}$ for $i = 1, \dots, n$ and $j = 1, 2, 3$.

(ii) $3SAT \leq_p VERTEXCOVER$

Proof:

► Intuition: Express 3SAT instance as a VERTEXCOVER instance.

► So, let φ be an arbitrary formula in 3-CNF over variables $x_1, \dots, x_m$

$\rightsquigarrow \varphi$ has the form $\underbrace{(l_{1,1} \vee l_{1,2} \vee l_{1,3})}_{C_1} \wedge \underbrace{(l_{2,1} \vee l_{2,2} \vee l_{2,3})}_{C_2} \wedge \dots \wedge \underbrace{(l_{n,1} \vee l_{n,2} \vee l_{n,3})}_{C_n},$

with $l_{i,j} \in \{x_1, \neg x_1, \dots, x_m, \neg x_m\}$ for $i = 1, \dots, n$ and $j = 1, 2, 3$.

► Define a graph $G = (V, E)$ via

$$V = \{L_{i,j} : i = 1, \dots, n; j = 1, 2, 3\}$$

$$E = \left\{ \{L_{i,j}, L_{p,q}\} : l_{i,j} \equiv \neg l_{p,q} \right\} \cup \left\{ \{L_{i,1}, L_{i,2}\}, \{L_{i,2}, L_{i,3}\}, \{L_{i,3}, L_{i,1}\} : i = 1, \dots, n \right\}$$

We “draw” a vertex for every literal of a clause. We connect them if

(a) they are literals in the same clause or (b) they are negations of each other

(ii) $3SAT \leq_p VERTEXCOVER$

Proof:

► Intuition: Express 3SAT instance as a VERTEXCOVER instance.

► So, let φ be an arbitrary formula in 3-CNF over variables $x_1, \dots, x_m$

$\rightsquigarrow \varphi$ has the form $\underbrace{(l_{1,1} \vee l_{1,2} \vee l_{1,3})}_{C_1} \wedge \underbrace{(l_{2,1} \vee l_{2,2} \vee l_{2,3})}_{C_2} \wedge \dots \wedge \underbrace{(l_{n,1} \vee l_{n,2} \vee l_{n,3})}_{C_n},$

with $l_{i,j} \in \{x_1, \neg x_1, \dots, x_m, \neg x_m\}$ for $i = 1, \dots, n$ and $j = 1, 2, 3$.

► Define a graph $G = (V, E)$ via

$$V = \{L_{i,j} : i = 1, \dots, n; j = 1, 2, 3\}$$

$$E = \left\{ \{L_{i,j}, L_{p,q}\} : l_{i,j} \equiv \neg l_{p,q} \right\} \cup \left\{ \{L_{i,1}, L_{i,2}\}, \{L_{i,2}, L_{i,3}\}, \{L_{i,3}, L_{i,1}\} : i = 1, \dots, n \right\}$$

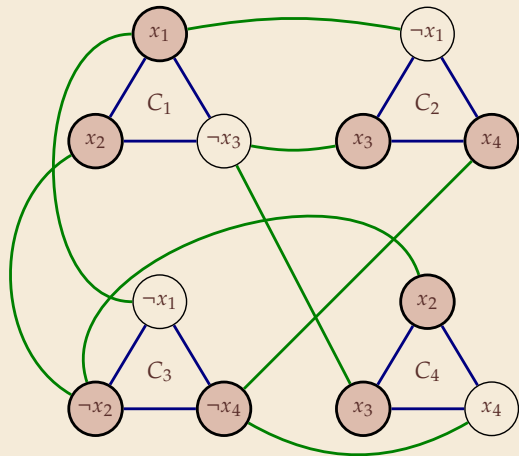
We “draw” a vertex for every literal of a clause. We connect them if

(a) they are literals in the same clause or (b) they are negations of each other

$\rightsquigarrow$ **Claim:** φ satisfiable $\iff G$ has vertex cover of size $\leq 2n$.

(ii) 3SAT $\leq_p$ VERTEXCOVER – Example

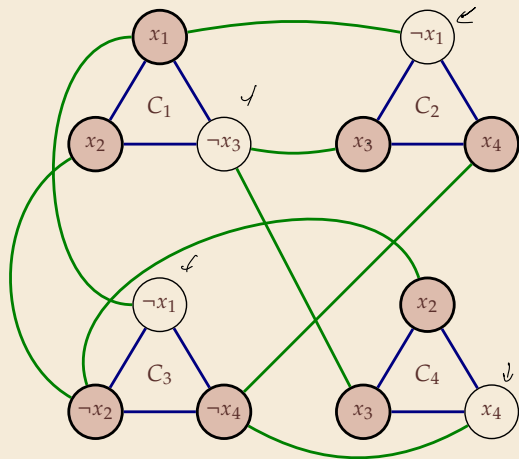
$$\varphi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_4) \wedge (x_2 \vee x_3 \vee x_4)$$



Set S (a VC of size $2n$)

(ii) 3SAT $\leq_p$ VERTEXCOVER – Example

$$\varphi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_4) \wedge (x_2 \vee x_3 \vee x_4)$$



Set S (a VC of size $2n$)

► **Idea:** Vertices *not in* vertex cover S define a variable assignment.

► Cannot be contradictory, otherwise “negation”-edge not covered.

► Must take ≥ 2 vertices per clause into S (otherwise triangle not covered)

$$\rightsquigarrow |S| \geq 2n \text{ for every vertex cover.}$$

► In the example:

► Fat vertices form a vertex cover for G

► corresponding assignment:

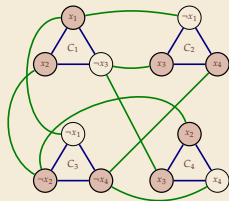
$$V = \{x_1 \mapsto 0, x_2 \mapsto 0, x_3 \mapsto 0, x_4 \mapsto 1\}$$

$(0 \hat{=} \text{false}, 1 \hat{=} \text{true})$

$\rightsquigarrow \varphi$ satisfiable

(ii) $3SAT \leq_p VERTEXCOVER$ – Correctness Proof

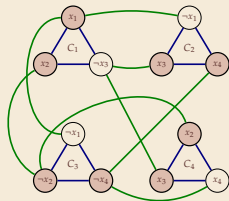
Claim: φ satisfiable $\iff G$ has VC S of size $|S| \leq 2n$.



(ii) $3SAT \leq_p VERTEXCOVER$ – Correctness Proof

Claim: φ satisfiable $\iff G$ has VC S of size $|S| \leq 2n$.

$\Rightarrow$ Let φ erfüllbar; $\rightsquigarrow$ every C_i has a satisfied literal $l_{i,j}$.



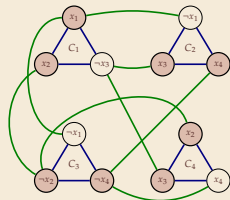
(ii) 3SAT $\leq_p$ VERTEXCOVER – Correctness Proof

Claim: φ satisfiable $\iff G$ has VC S of size $|S| \leq 2n$.

$\Rightarrow$ Let φ erfüllbar; $\rightsquigarrow$ every C_i has a satisfied literal $l_{i,j}$.

Add the *other two* vertices of the clause to S .

$\rightsquigarrow |S| = 2n$ and S covers all clause triangle edges.



(ii) 3SAT $\leq_p$ VERTEXCOVER – Correctness Proof

Claim: φ satisfiable $\iff G$ has VC S of size $|S| \leq 2n$.

$\Rightarrow$ Let φ erfüllbar; $\rightsquigarrow$ every C_i has a satisfied literal $l_{i,j}$.

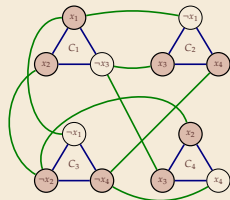
Add the *other two* vertices of the clause to S .

$\rightsquigarrow |S| = 2n$ and S covers all **clause triangle edges**.

Remaining edges have the form $\{x, \neg x\}$.

If such an edge remained uncovered by S , we would have that both x and $\neg x$ are satisfied literals $\nmid$

$\rightsquigarrow G$ has VC of size $2n$.



(ii) 3SAT $\leq_p$ VERTEXCOVER – Correctness Proof

Claim: φ satisfiable $\iff G$ has VC S of size $|S| \leq 2n$.

$\Rightarrow$ Let φ erfüllbar; $\rightsquigarrow$ every C_i has a satisfied literal $l_{i,j}$.

Add the *other two* vertices of the clause to S .

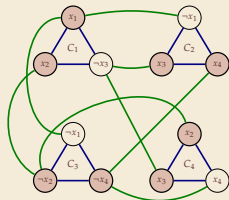
$\rightsquigarrow |S| = 2n$ and S covers all clause triangle edges.

Remaining edges have the form $\{x, \neg x\}$.

If such an edge remained uncovered by S , we would have that both x and $\neg x$ are satisfied literals $\downarrow$

$\rightsquigarrow$ G has VC of size $2n$.

$\Leftarrow$ Given a VC S of G with $|S| \leq 2n$.



(ii) 3SAT $\leq_p$ VERTEXCOVER – Correctness Proof

Claim: φ satisfiable $\iff G$ has VC S of size $|S| \leq 2n$.

$\Rightarrow$ Let φ erfüllbar; $\rightsquigarrow$ every C_i has a satisfied literal $l_{i,j}$.

Add the *other two* vertices of the clause to S .

$\rightsquigarrow |S| = 2n$ and S covers all **clause triangle edges**.

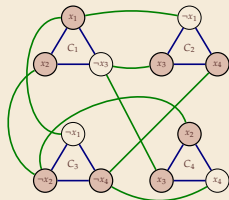
Remaining edges have the form $\{x, \neg x\}$.

If such an edge remained uncovered by S , we would have that both x and $\neg x$ are satisfied literals $\nmid$

$\rightsquigarrow G$ has VC of size $2n$.

$\Leftarrow$ Given a VC S of G with $|S| \leq 2n$.

S must contain 2 vertices per **clause triangle** $\rightsquigarrow |S| = 2n$ and S is a *minimal* VC.



(ii) 3SAT $\leq_p$ VERTEXCOVER – Correctness Proof

Claim: φ satisfiable $\iff G$ has VC S of size $|S| \leq 2n$.

$\Rightarrow$ Let φ erfüllbar; $\rightsquigarrow$ every C_i has a satisfied literal $l_{i,j}$.

Add the *other two* vertices of the clause to S .

$\rightsquigarrow |S| = 2n$ and S covers all **clause triangle edges**.

Remaining edges have the form $\{x, \neg x\}$.

If such an edge remained uncovered by S , we would have that both x and $\neg x$ are satisfied literals $\nmid$

$\rightsquigarrow G$ has VC of size $2n$.

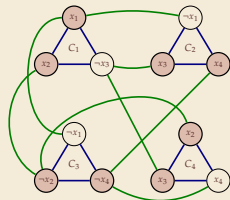
$\Leftarrow$ Given a VC S of G with $|S| \leq 2n$.

S must contain 2 vertices per **clause triangle** $\rightsquigarrow |S| = 2n$ and S is a *minimal* VC.

Define assignment V so that all literals *not* in S are satisfied.

(Variables which are not assigned a value via this procedure can be assigned an arbitrary value.)

V is well defined, since $\{x, \neg x\}$ -edges must be covered.



(ii) $3SAT \leq_p VERTEXCOVER$ – Correctness Proof

Claim: φ satisfiable $\iff G$ has VC S of size $|S| \leq 2n$.

$\Rightarrow$ Let φ erfüllbar; $\rightsquigarrow$ every C_i has a satisfied literal $l_{i,j}$.

Add the *other two* vertices of the clause to S .

$\rightsquigarrow |S| = 2n$ and S covers all **clause triangle edges**.

Remaining edges have the form $\{x, \neg x\}$.

If such an edge remained uncovered by S , we would have that both x and $\neg x$ are satisfied literals $\nmid$

$\rightsquigarrow G$ has VC of size $2n$.

$\Leftarrow$ Given a VC S of G with $|S| \leq 2n$.

S must contain 2 vertices per **clause triangle** $\rightsquigarrow |S| = 2n$ and S is a *minimal* VC.

Define assignment V so that all literals *not* in S are satisfied.

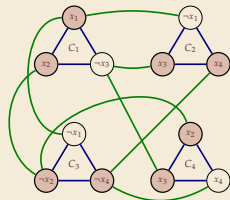
(Variables which are not assigned a value via this procedure can be assigned an arbitrary value.)

V is well defined, since $\{x, \neg x\}$ -edges must be covered.

Moreover, V makes φ true:

from every clause, at least one literal is satisfied since $|S| = 2n$.

$\rightsquigarrow \varphi$ satisfiable.



(ii) $3\text{SAT} \leq_p \text{VERTEXCOVER}$ – Running Time

- ▶ Construction of G upon input φ can easily be done in polytime
 - ▶ $|V| = O(n), |E| = O(n^2)$
 - ▶ Construction of E in time $O(n^2)$ easy to do, e. g., on RAM $\rightsquigarrow \exists$ polytime TM.

$\rightsquigarrow 3\text{SAT} \leq_p \text{VERTEXCOVER}.$

2.8 Important NP-Complete Problems

Further NP-complete problems [1]

Apart from SAT, 3SAT, and VERTEXCOVER, here are some of the most useful NP-complete problems.

Definition 2.29 (Dominating Set)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Question: $\exists V' \subset V : |V'| \leq k \wedge \forall v \in V : (v \in V' \vee \exists u \in N(v) : u \in V')$



Definition 2.30 (Hamiltonian Cycle)

Given: graph $G = (V, E)$ (directed and undirected version)

Question: Is there a vertex-simple cycle in G of length $|V|$?



Definition 2.31 (Clique)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Question: $\exists V' \subset V : |V'| \geq k \wedge \forall u, v \in V' : \{u, v\} \in E$



Definition 2.32 (Independent Set)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Question: $\exists V' \subset V : |V'| \geq k \wedge \forall u, v \in V' : \{u, v\} \notin E$



Further NP-complete problems [2]

Definition 2.33 (Traveling Salesperson (TSP))

Given: distance matrix $D \in \mathbb{N}^{n \times n}$ and $k \in \mathbb{N}$

Question: Is there a permutation $\pi : [n] \rightarrow [n]$ with $\sum_{i=1}^{n-1} D_{\pi(i), \pi(i+1)} + D_{\pi(n), \pi(1)} \leq k$?

Definition 2.34 (Graph Coloring)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Question: $\exists c : V \rightarrow [k] : \forall \{u, v\} \in E : c(u) \neq c(v)$?

Definition 2.35 (Set Cover)

Given: $n \in \mathbb{N}$, sets $S_1, \dots, S_m \subseteq [n]$ and $k \in \mathbb{N}$

Question: $\exists I \subseteq [m] : \bigcup_{i \in I} S_i = [n] \wedge |I| \leq k$?

Definition 2.36 (Weighted Set Cover)

Given: $n \in \mathbb{N}$, sets $S_1, \dots, S_m \subseteq [n]$, costs $c_1, \dots, c_m \in \mathbb{N}_0$ and $k \in \mathbb{N}$

Question: $\exists I \subseteq [m] : \bigcup_{i \in I} S_i = [n] \wedge \sum_{i \in I} c_i \leq k$?

Further hard problems [3]

Definition 2.37 (Closest String)

Given: $s_1, \dots, s_n \in \Sigma^m$ and $k \in \mathbb{N}$

Question: $\exists s \in \Sigma^m : \forall i \in [n] : d_H(s, s_i) \leq k ?$ (d_H Hamming-distance) ◀

Definition 2.38 (Max Cut)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Question: $\exists C \subset V : |E \cap \{\{u, v\} \mid u \in C, v \notin C\}| \geq k ?$ ◀

Definition 2.39 (Exact Cover)

Given: $n \in \mathbb{N}$, sets $S_1, \dots, S_m \subseteq [n]$

Question: $\exists I \subseteq [m] : \bigcup_{i \in I} S_i = [n] \wedge \sum_{i \in I} |S_i| = n ?$ ◀

Further hard problems [4]

Definition 2.40 (Subset Sum)

Given: $x_1, \dots, x_n \in \mathbb{Z}$

Question: $\exists I \subseteq [n] : I \neq \emptyset \wedge \sum_{i \in I} x_i = 0$?

Definition 2.41 ((0/1) Knapsack)

Given: $w_1, \dots, w_n \in \mathbb{N}, v_1, \dots, v_n \in \mathbb{N}$ and $b, k \in \mathbb{N}$

Question: $\exists I \subseteq [n] : \sum_{i \in I} w_i \leq b \wedge \sum_{i \in I} v_i \geq k$?

Definition 2.42 (Bin Packing)

Given: $w_1, \dots, w_n \in \mathbb{N}, b \in \mathbb{N}, k \in \mathbb{N}$

Question: $\exists a : [n] \rightarrow [k] : \forall j \in [k] : \sum_{\substack{i=1, \dots, n \\ a[i]=j}} w_i \leq b$?

Definition 2.43 (0/1 Integer Programming)

Given: integer linear program (ILP) $A \in \mathbb{Z}^{m \times n}, b \in \mathbb{Z}^m$ and $c \in \mathbb{Z}^n$ and $k \in \mathbb{Z}$

Question: Is there $x \in \{0, 1\}^n$ with $Ax \leq b$ and $c^T x \geq k$?

2.9 Optimization Problems

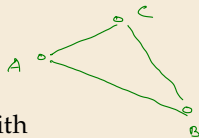
Optimization Problems

Definition 2.44 (Optimization Problem)

An *optimization problem* is given by 7-tuple $U = (\Sigma_I, \Sigma_O, L, L_I, M, cost, goal)$ with

1. Σ_I an alphabet (called input alphabet),
2. Σ_O an alphabet (called output alphabet),
3. $L \subseteq \Sigma_I^*$ the language of **allowable** problem instances (for which U is well-defined),
4. $L_I \subseteq L$ the language of **actual** problem instances for U (for those we want to determine U 's complexity),
5. $M : L \rightarrow 2^{\Sigma_O^*}$ and with $x \in L$, $M(x)$ is the set of all **feasible** solutions for x .
6. $cost$ is a cost function, which assigns for $x \in L$ each pair (u, x) with $u \in M(x)$ a positive real number,
7. $goal \in \{\min, \max\}$.

metric TSP



$$D(A, B) \leq D(A, C) + D(C, B)$$

distance matrices

metric distance matrices

permutation
of cities

Optimal Solutions

Definition 2.45 (Optimal Solutions, Solution Algorithms)

Let $U = (\Sigma_I, \Sigma_O, L, L_I, M, cost, goal)$ an optimization problem. For each $x \in L_I$ a feasible solution $y \in M(x)$ is called *optimal for x and U* , if

$$cost(y, x) = goal\{cost(z, x) \mid z \in M(x)\}.$$

An algorithm A is *consistent with U* if $A(x) \in M(x)$ for all $x \in L_I$.

We say *algorithm B solves U* , if

1. B is consistent with U and
2. for all $x \in L_I$, $B(x)$ is optimal for x and U .

Optimization Problems – Examples

Natural examples: Problems above with an input parameter k .

Less immediate example:

$$C_i = (x_i \vee \neg x_j \vee x_k \dots)$$

Definition 2.46 (Max-SAT)

Given: CNF-Formula $\phi = C_1 \wedge \dots \wedge C_m$ over variables $x_1, \dots, x_n$

Allowable (=Actual) Instances: encodings of ϕ

$M(\phi) = \{0, 1\}^n$ (variable assignments)

$cost(u, x)$: # of satisfied clauses in u under given assignment x

$goal = \max$



Classes of Optimization Problems

Definition 2.47 (NPO)

NPO is the class of optimization problems $U = (\Sigma_I, \Sigma_O, L, L_I, M, cost, goal)$ with

1. $L_I \in P$,
2. there is a polynomial p_U with
 - a) $\forall x \in L_I \forall y \in M(x) : |y| \leq p_U(|x|)$ and
 - b) there is a polynomial time algorithm which for all $y \in \Sigma_O^*, x \in L_I$ with $|y| \leq p_U(|x|)$ decides whether $y \in M(x)$ holds, and
3. function *cost* can be computed in polynomial time. 

Classes of Optimization Problems

Definition 2.47 (NPO)

NPO is the class of optimization problems $U = (\Sigma_I, \Sigma_O, L, L_I, M, cost, goal)$ with

1. $L_I \in P$,
2. there is a polynomial p_U with
 - a) $\forall x \in L_I \forall y \in M(x) : |y| \leq p_U(|x|)$ and
 - b) there is a polynomial time algorithm which for all $y \in \Sigma_O^*, x \in L_I$ with $|y| \leq p_U(|x|)$ decides whether $y \in M(x)$ holds, and
3. function *cost* can be computed in polynomial time. 

Definition 2.48 (PO)

PO is the class of optimization problems $U = (\Sigma_I, \Sigma_O, L, L_I, M, cost, goal)$ with

1. $U \in NPO$, and
2. there is an algorithm of polynomial time complexity which for all $x \in L_I$ computes an optimal solution for x and U . 

Glossary of Problem Types

	check threshold	find optimal value
single value	decision problems (languages)	evaluation problems
solution/ certificate	search problems (VP)	optimization problems

↖ (cost ∈ P)
easy

↖
easy

- ↗ threshold to opt? : binary search ✓ cost ∈ M
- ↘ decision to search : unclear in general -- often possible

From Optimization to Decision

Definition 2.49 (Threshold Languages)

Let $U = (\Sigma_I, \Sigma_O, L, L_I, M, cost, goal)$ an optimization problem, $U \in \text{NPO}$.

For $Opt_U(x)$ the **cost** of an optimal solutions for x and U we define the *threshold language for U* as

$$Lang_U = \begin{cases} \{(x, k) \in L_I \times \{0, 1\}^* \mid Opt_U(x) \leq k_2\}, & \text{if } goal = \min, \\ \{(x, k) \in L_I \times \{0, 1\}^* \mid Opt_U(x) \geq k_2\}, & \text{if } goal = \max. \end{cases}$$

k as binary number

We say U is *NP-hard*, if $Lang_U$ is NP-hard.



Hardness of Optimization

Corollary 2.50 (Optimization is harder than Threshold)

Let U an optimization problem.

If $Lang_U$ is NP-hard and if $P \neq NP$ holds, we have $U \notin PO$.



Hardness of Optimization

Corollary 2.50 (Optimization is harder than Threshold)

Let U an optimization problem.

If $Lang_U$ is NP-hard and if $P \neq NP$ holds, we have $U \notin PO$.

Proof:

Contraposition. Suppose $U \in PO$.



Hardness of Optimization

Corollary 2.50 (Optimization is harder than Threshold)

Let U an optimization problem.

If $Lang_U$ is NP-hard and if $P \neq NP$ holds, we have $U \notin PO$.

Proof:

Contraposition. Suppose $U \in PO$.

$\rightsquigarrow \exists$ polytime algo A that computes optimal y for any instance x of U



Hardness of Optimization

Corollary 2.50 (Optimization is harder than Threshold)

Let U an optimization problem.

If $Lang_U$ is NP-hard and if $P \neq NP$ holds, we have $U \notin PO$.

Proof:

Contraposition. Suppose $U \in PO$.

- $\rightsquigarrow \exists$ polytime algo A that computes optimal y for any instance x of U
- $\rightsquigarrow$ can compute $Opt_U(x) = cost(y)$ in polytime

Hardness of Optimization

Corollary 2.50 (Optimization is harder than Threshold)

Let U an optimization problem.

If $Lang_U$ is NP-hard and if $P \neq NP$ holds, we have $U \notin PO$.

Proof:

Contraposition. Suppose $U \in PO$.

- $\rightsquigarrow \exists$ polytime algo A that computes optimal y for any instance x of U
- $\rightsquigarrow$ can compute $Opt_U(x) = cost(y)$ in polytime
- $\rightsquigarrow$ can decide whether $Opt_U(x)$ better than threshold in polytime

Hardness of Optimization

Corollary 2.50 (Optimization is harder than Threshold)

Let U an optimization problem.

If $Lang_U$ is NP-hard and if $P \neq NP$ holds, we have $U \notin PO$.

Proof:

Contraposition. Suppose $U \in PO$.

- $\rightsquigarrow \exists$ polytime algo A that computes optimal y for any instance x of U
- $\rightsquigarrow$ can compute $Opt_U(x) = cost(y)$ in polytime
- $\rightsquigarrow$ can decide whether $Opt_U(x)$ better than threshold in polytime
- $\rightsquigarrow Lang_U \in P$.

Hardness of Optimization

Corollary 2.50 (Optimization is harder than Threshold)

Let U an optimization problem.

If $Lang_U$ is NP-hard and if $P \neq NP$ holds, we have $U \notin PO$.

Proof:

Contraposition. Suppose $U \in PO$.

$\rightsquigarrow \exists$ polytime algo A that computes optimal y for any instance x of U

$\rightsquigarrow$ can compute $Opt_U(x) = cost(y)$ in polytime

$\rightsquigarrow$ can decide whether $Opt_U(x)$ better than threshold in polytime

$\rightsquigarrow Lang_U \in P$.

⚡ $Lang_U$ NP-hard $\rightsquigarrow U \notin PO$.

Max-SAT is hard

Corollary 2.51 (Max-SAT is hard)

Max-SAT is NP-hard.



Max-SAT is hard

Corollary 2.51 (Max-SAT is hard)

MAX-SAT is NP-hard.

Proof:

We show: $3\text{SAT} \leq_p \text{Lang}_{\text{MAX-SAT}}$.

Given a 3SAT instance x encoding a formula with m clauses, output (x, m) .



Max-SAT is hard

Corollary 2.51 (Max-SAT is hard)

MAX-SAT is NP-hard.

Proof:

We show: $3SAT \leq_p Lang_{MAX-SAT}$.

Given a 3SAT instance x encoding a formula with m clauses, output (x, m) .

$x \in 3SAT \iff x$ is satisfiable $\iff m$ clauses of x satisfiable $\iff (x, m) \in Lang_{MAX-SAT}$. ■

Summary

- ▶ We have formalized the classic notion of intractable problems.
 - ▶ What is running time, what is “polytime”?
 - ▶ Decision problems $\leftrightarrow$ (formal) languages
 - ▶ P, NP via Turing machines $\leftrightarrow$ certificates and verifiers
 - ▶ For the typical case of optimization problems, there are different versions of the problem, but (in)tractability typically carries over.
- $\rightsquigarrow$ We can mathematically prove a problem is intractable (NP-hard).

Summary

► We have formalized the classic notion of intractable problems.

► What is running time, what is “polytime”?

► Decision problems $\leftrightarrow$ (formal) languages

► P, NP via Turing machines $\leftrightarrow$ certificates and verifiers

► For the typical case of optimization problems,
there are different versions of the problem,
but (in)tractability typically carries over.

$\rightsquigarrow$ We can mathematically prove a problem is intractable (NP-hard).

... but how can we tackle hard problems anyway?